距离矩阵计算与K-means聚类

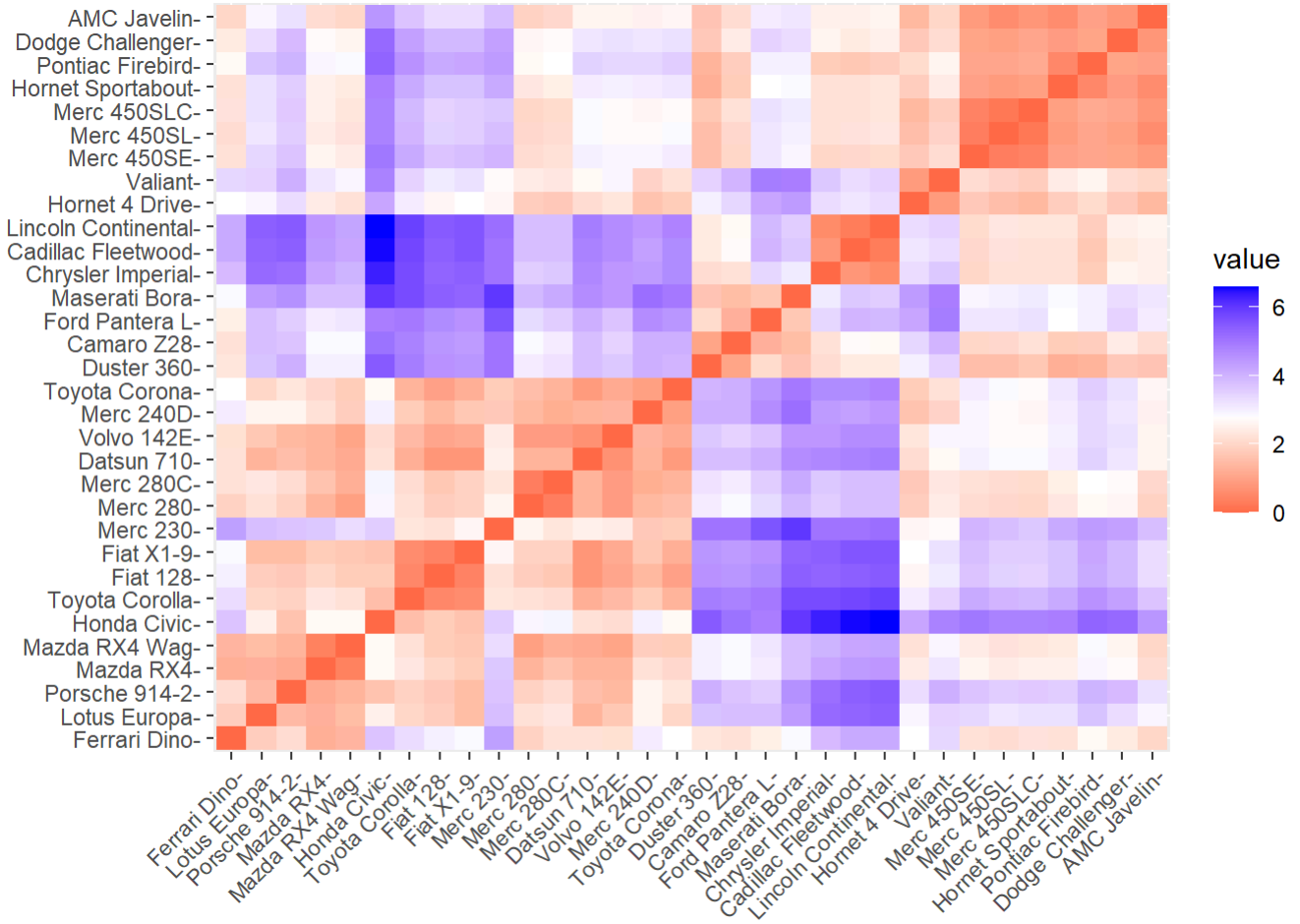
localhost:5335

王康

距离矩阵可视化

热图展示距离矩阵

```
fviz_dist(dist2)
```

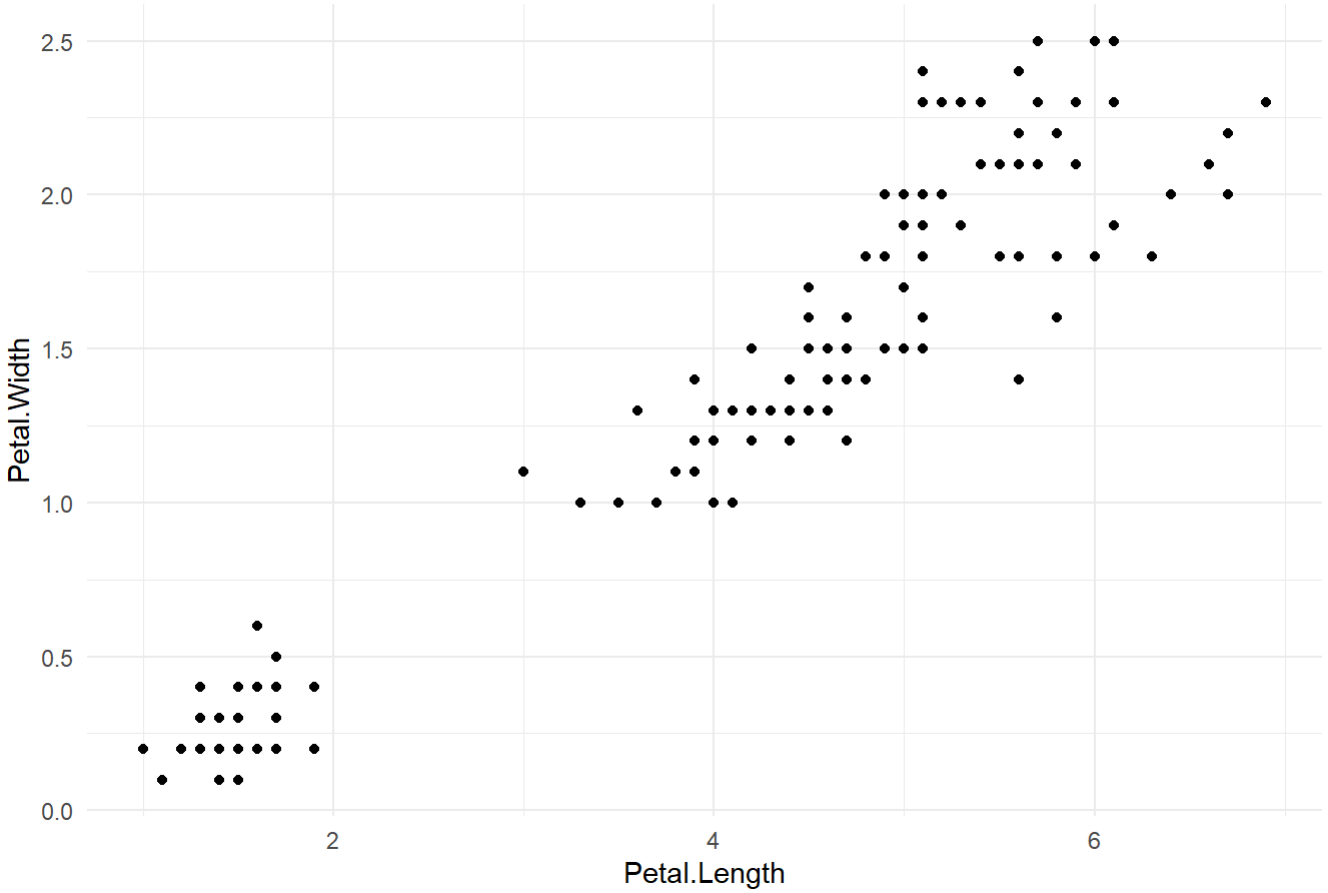


K-means

K值选择方法

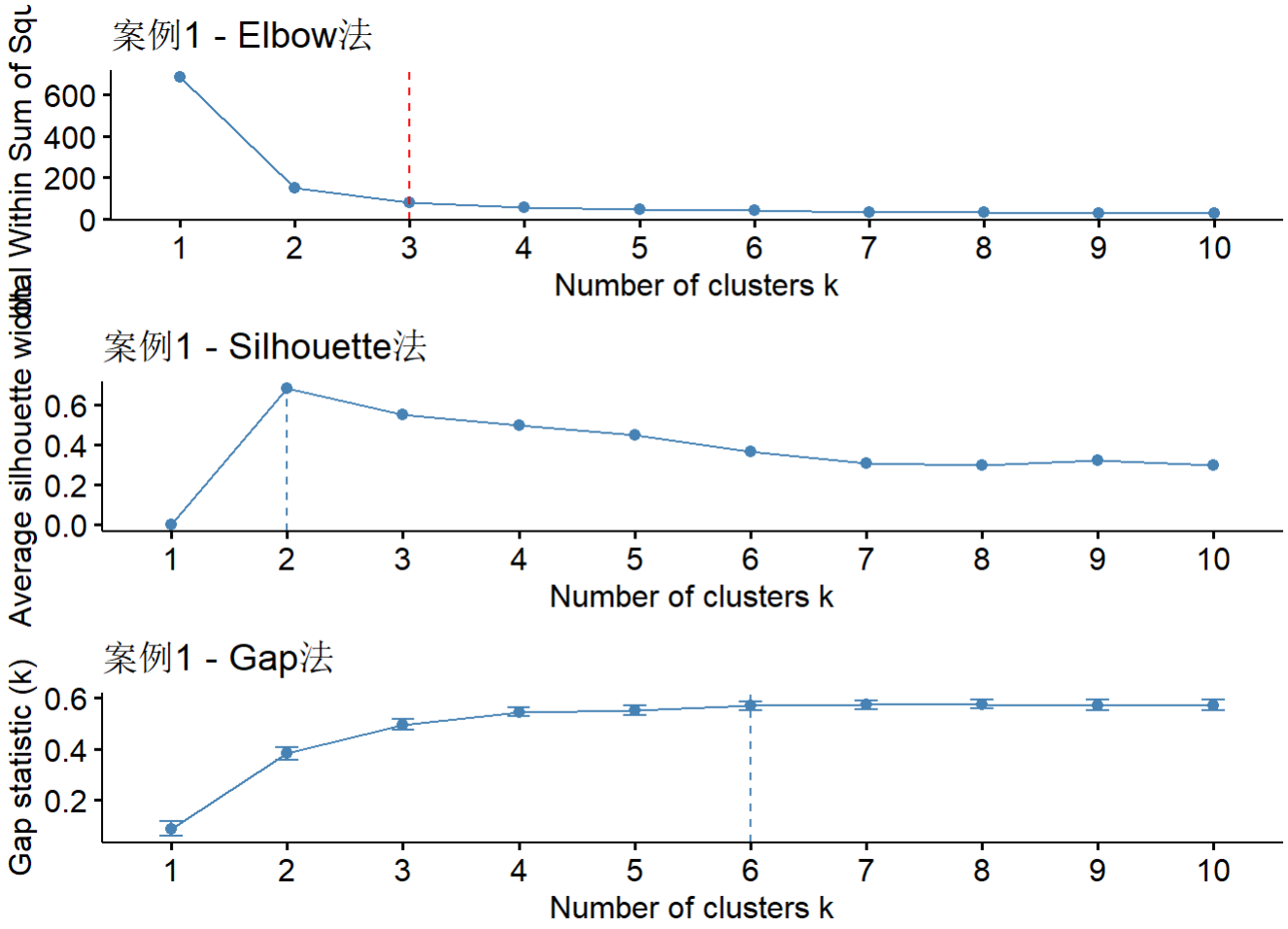
```
# 确保你安装了这些包
# install.packages("tidyverse")
# install.packages("factoextra") # 我们的可视化和评估主力
# install.packages("cluster")    # 支持 gap statistic 和 silhouette
# install.packages("MASS")       # 用于生成案例2的数据

library(tidyverse)
library(factoextra) # 核心包
library(cluster)
library(MASS) # 加载 MASS 包

# 设定一个随机种子，确保你的结果可以复现
set.seed(123)

# 使用 iris 数据，移除“物种”列 (第五列)，因为 K-Means 是无监督的
data_clear <- iris %>%
  as_tibble() %>%
  dplyr::select(-Species)

# （可选）在分享时展示：数据长这样，肉眼可见三簇
ggplot(data_clear, aes(x = Petal.Length, y = Petal.Width)) +
  geom_point() +
  theme_minimal() +
  labs(title = "案例1: 清晰的聚类 (Iris 数据)")
```

案例1: 清晰的聚类 (Iris 数据)



直接使用

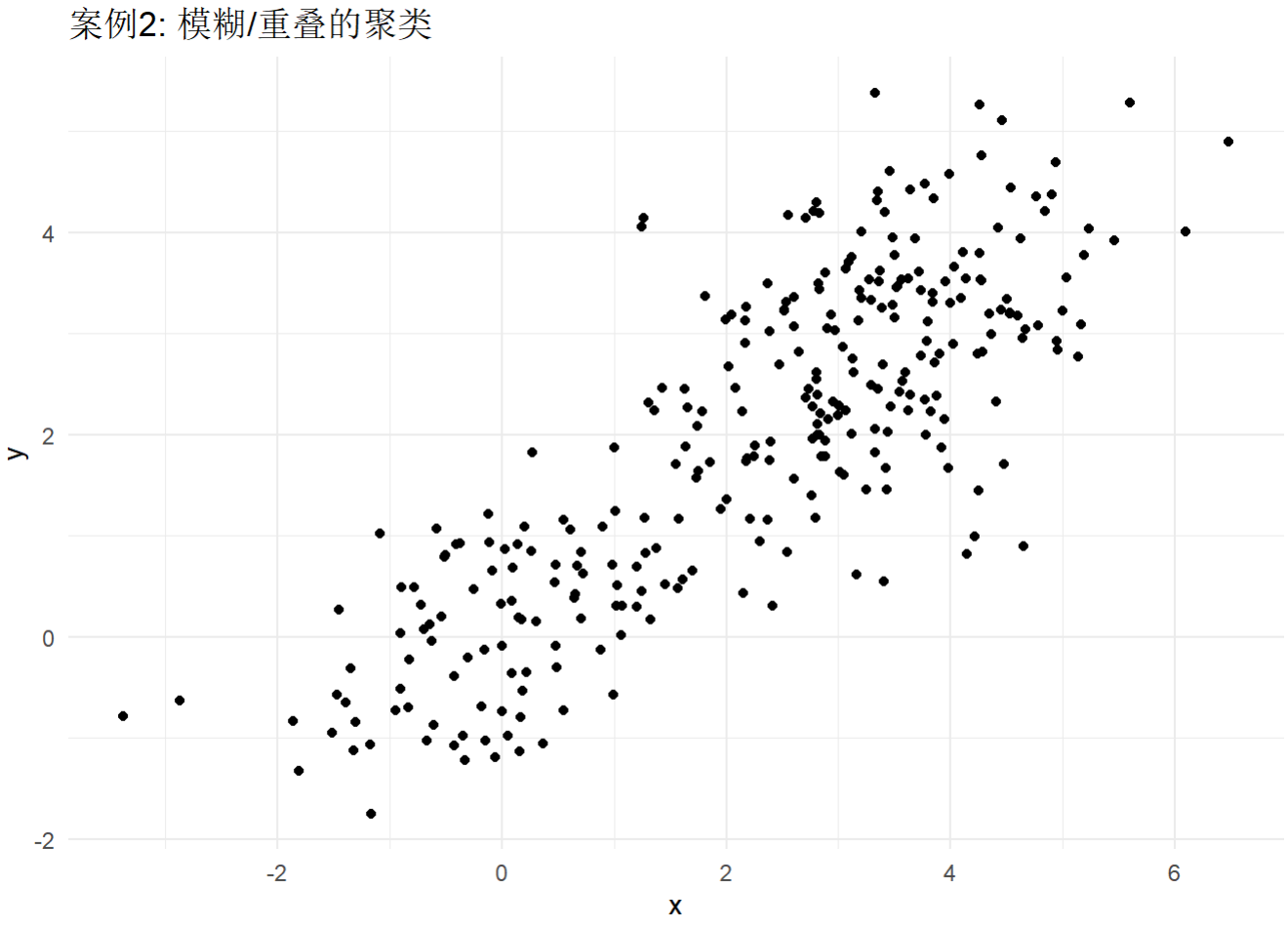
```
# 使用肘部法则 (Elbow Method) 选择 K 值
# 1. Elbow (肘部) 法
# "wss" = "Total Within Sum of Squares"
p_elbow_1 <- fviz_nbclust(data_clear, kmeans, method = "wss") +
  labs(title = "案例1 - Elbow法") +
  geom_vline(xintercept = 3, linetype = 2, color = "red") # 最佳k

# 2. Silhouette (轮廓) 法
p_sil_1 <- fviz_nbclust(data_clear, kmeans, method = "silhouette") +
  labs(title = "案例1 - Silhouette法")
# 注意：fviz_nbclust 会自动帮你找到最大值点 (k=3)

# 3. Gap 统计量法
# 注意: nboot = 50 是为了演示速度。
# 在你的真实论文中，请使用默认值 (nboot = 500) 或更高，这会非常慢！
p_gap_1 <- fviz_nbclust(
  data_clear,
  kmeans,
  method = "gap_stat",
  nboot = 50,
  nstart = 25
) +
  labs(title = "案例1 - Gap法")
# 注意：fviz_nbclust 会自动帮你找到最大值点 (k=3)

# (可选) 把图组合起来看
gridExtra::grid.arrange(p_elbow_1, p_sil_1, p_gap_1, ncol = 1)
```



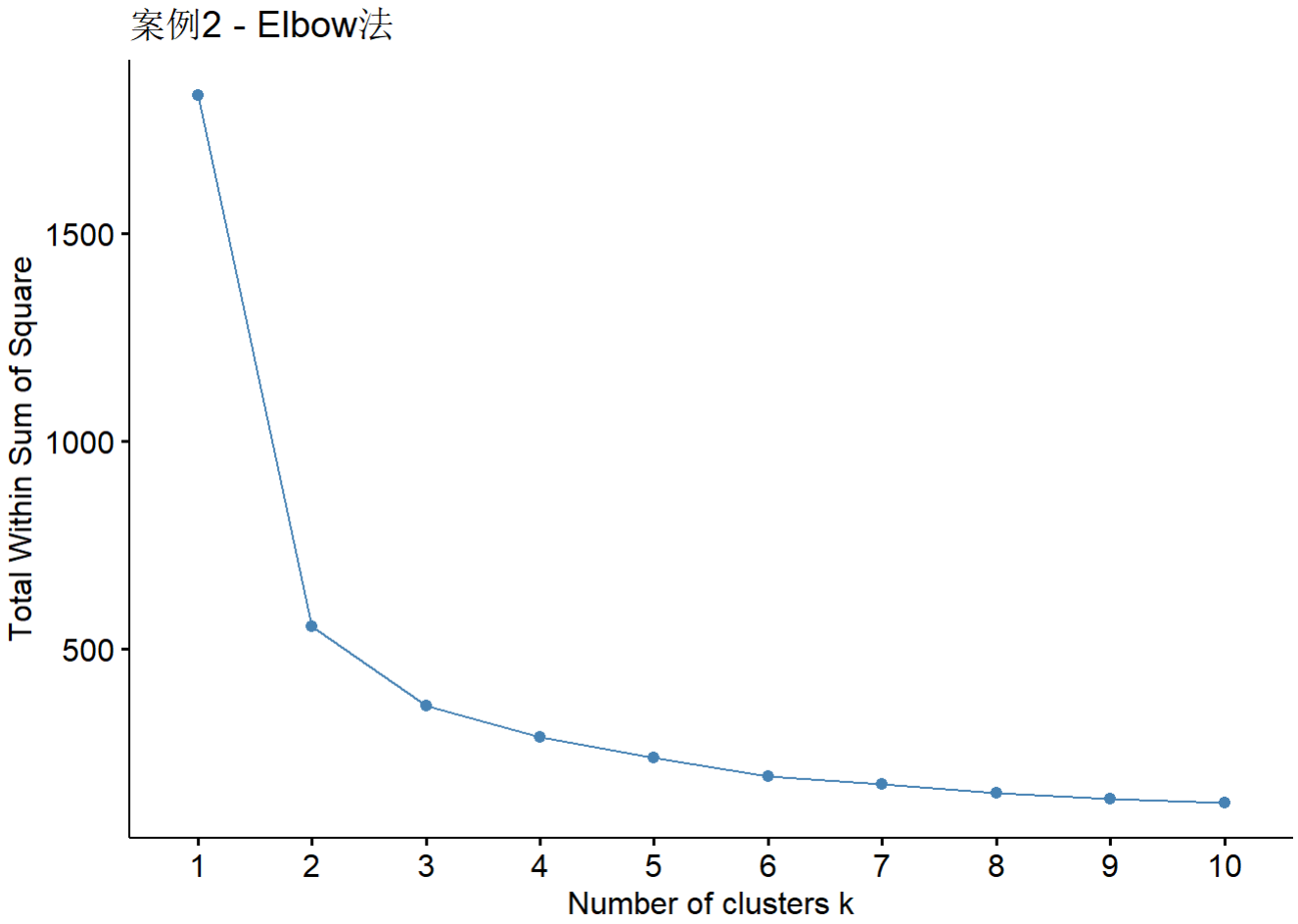

```
# 或者使用 patchwork 包`
```

模糊案例

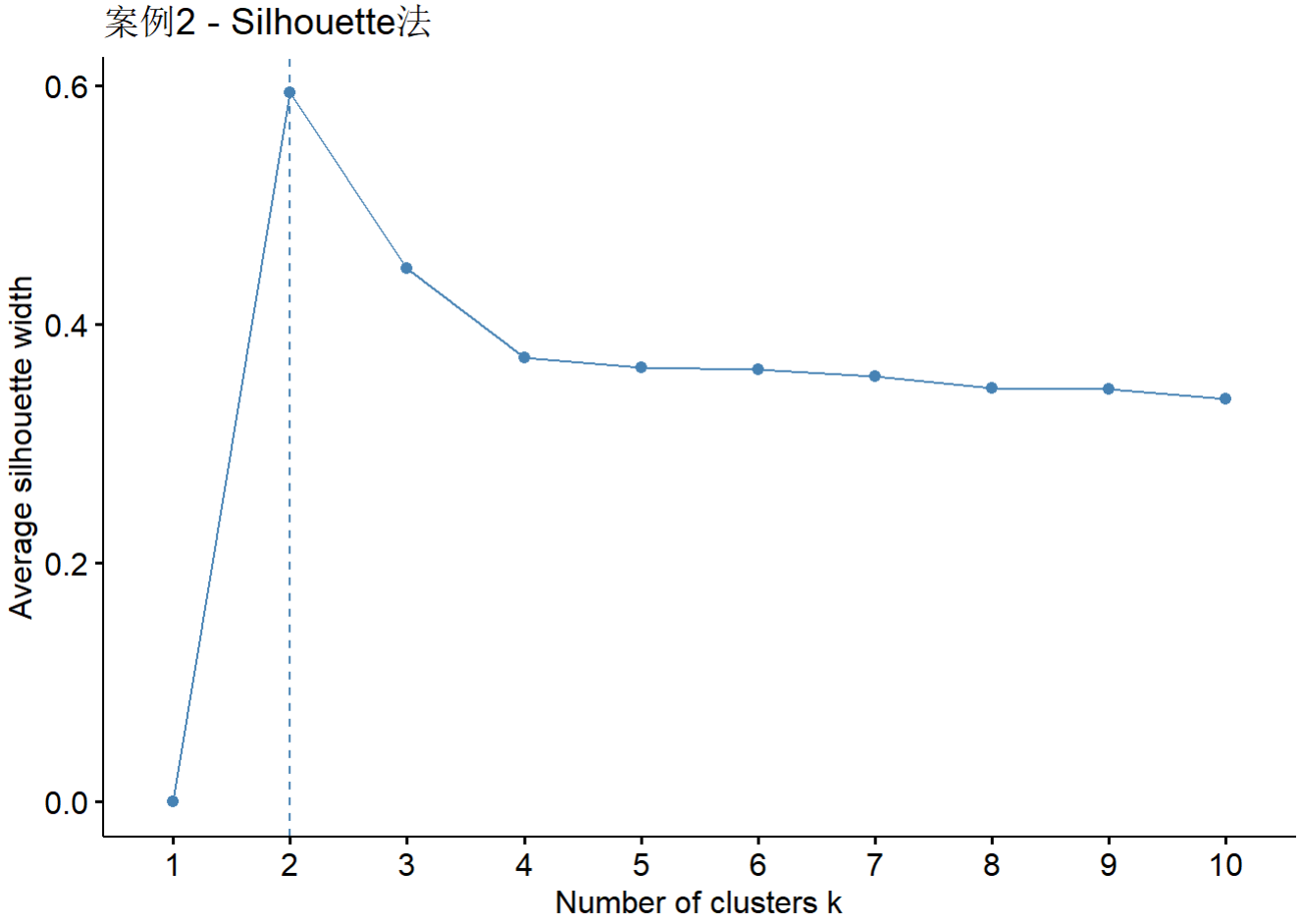
```
# 定义簇的中心和协方差矩阵
mu1 <- c(0, 0)
mu2 <- c(3, 3)
mu3 <- c(3.5, 2.5) # 注意：簇2和簇3离得非常近
sigma <- matrix(c(1, 0.5, 0.5, 1), 2)

# 生成数据
g1 <- mvrnorm(n = 100, mu = mu1, Sigma = sigma)
g2 <- mvrnorm(n = 100, mu = mu2, Sigma = sigma)
g3 <- mvrnorm(n = 100, mu = mu3, Sigma = sigma)

data_fuzzy <- bind_rows(
  as_tibble(g1, .name_repair = "minimal"),
  as_tibble(g2, .name_repair = "minimal"),
  as_tibble(g3, .name_repair = "minimal")
)
names(data_fuzzy) <- c("x", "y")

# (可选) 展示数据：簇2和簇3几乎融为一体
ggplot(data_fuzzy, aes(x, y)) +
  geom_point() +
  theme_minimal() +
  labs(title = "案例2: 模糊/重叠的聚类")
```



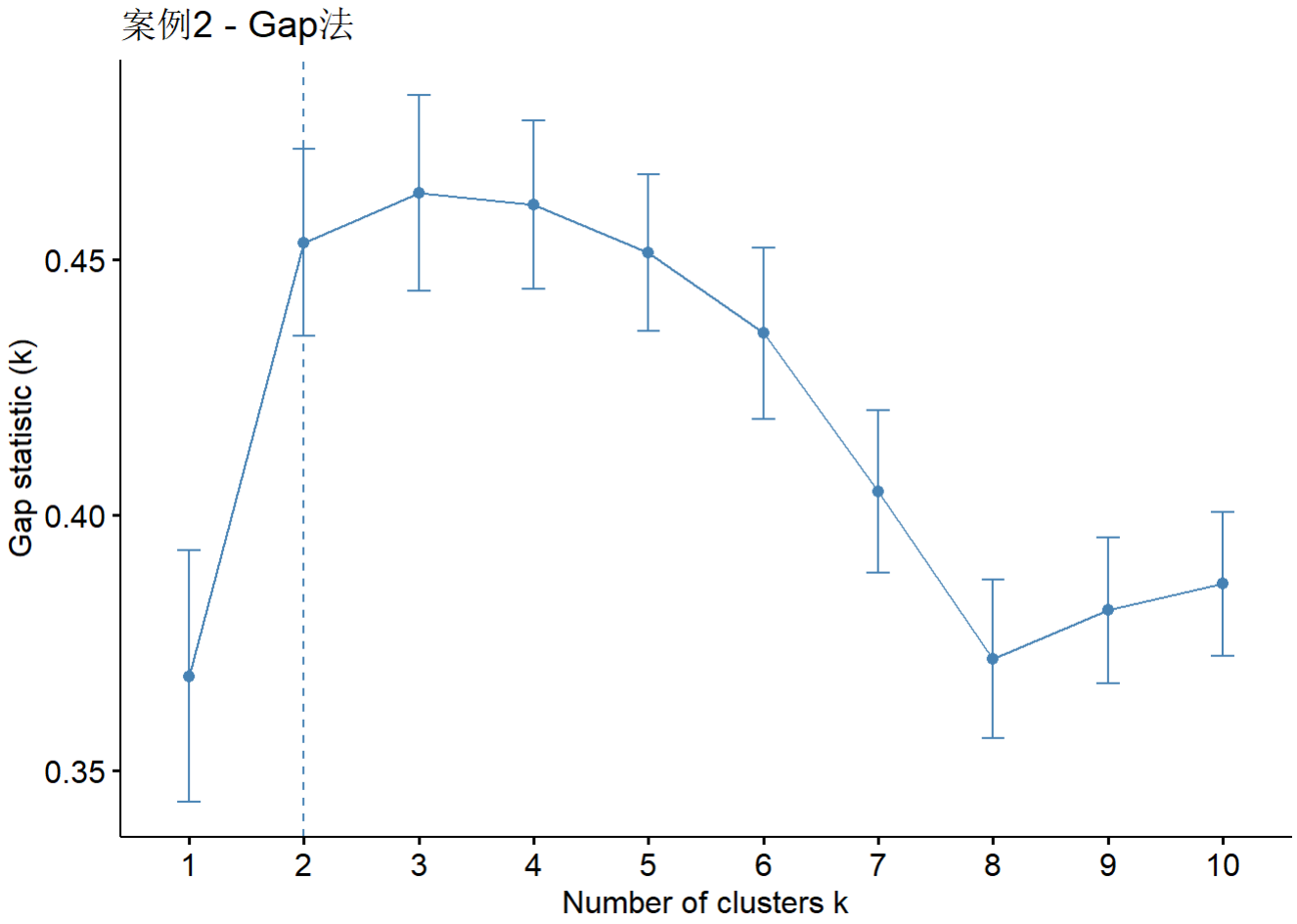
```
# 1. Elbow 法
p_elbow_2 <- fviz_nbclust(data_fuzzy, kmeans, method = "wss") +
  labs(title = "案例2 - Elbow法")

p_elbow_2
```




```
# 注意看：肘点在哪里？ k=2? k=3?

# 2. Silhouette 法
p_sil_2 <- fviz_nbclust(data_fuzzy, kmeans, method = "silhouette") +
  labs(title = "案例2 - Silhouette法")

p_sil_2
```



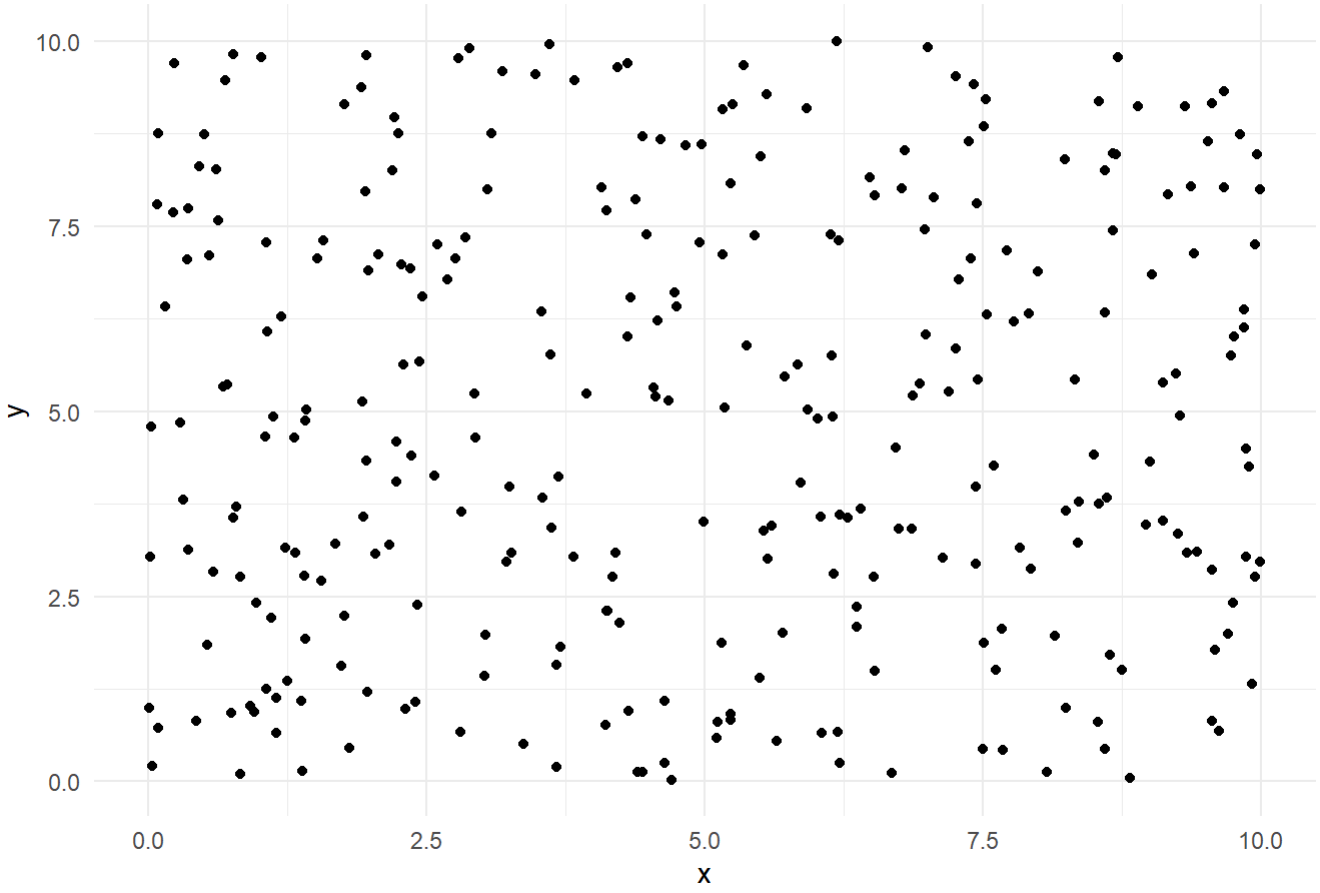

```
# 峰值在 k=2 还是 k=3？

# 3. Gap 统计量法
p_gap_2 <- fviz_nbclust(
  data_fuzzy,
  kmeans,
  method = "gap_stat",
  nboot = 50,
  nstart = 25
) +
  labs(title = "案例2 - Gap法")
p_gap_2
```



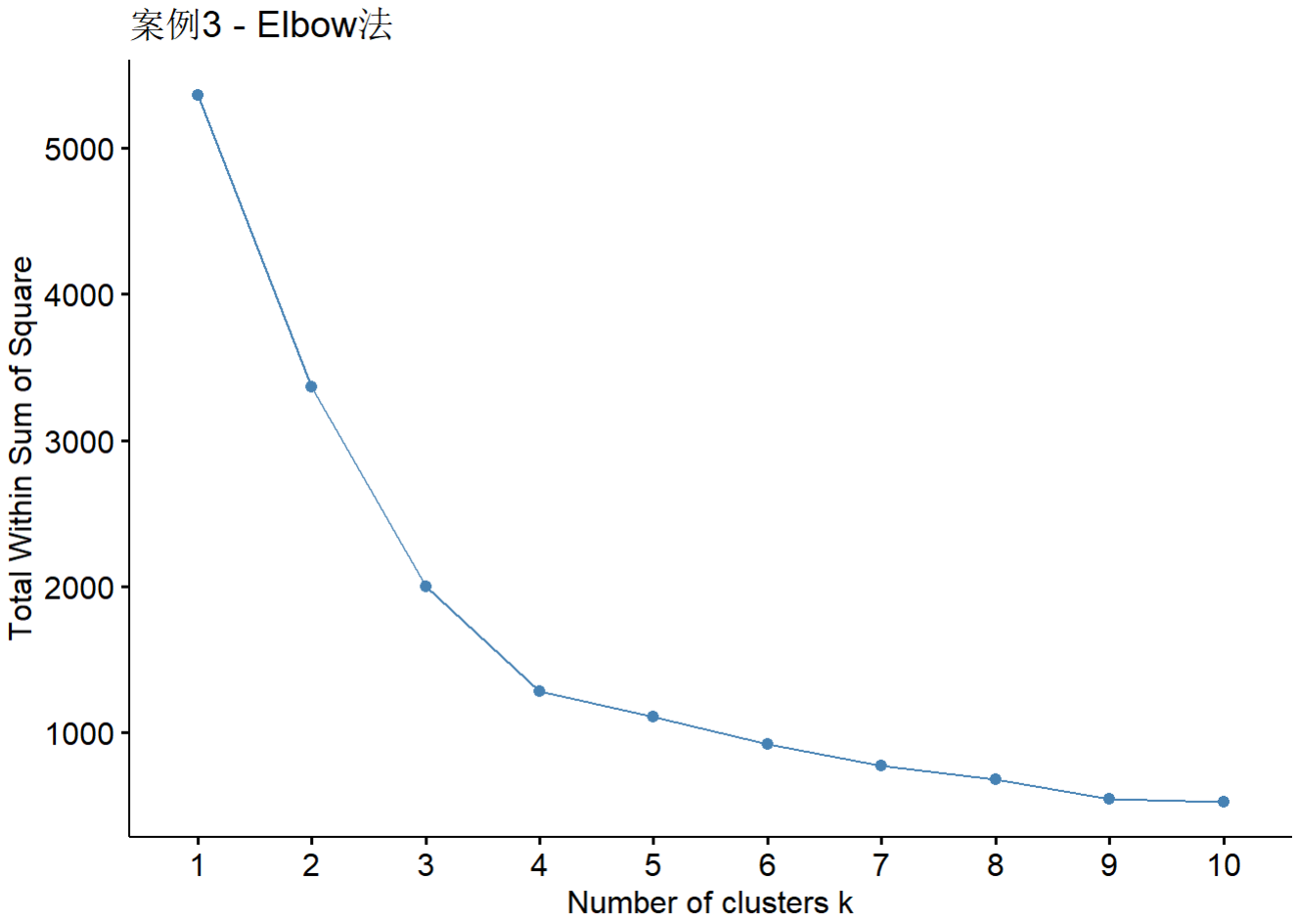
随机数据

```
# 生成均匀分布的随机数据
data_random <- tibble(
  x = runif(300, min = 0, max = 10),
  y = runif(300, min = 0, max = 10)
)

# (可选) 展示数据：完全没有结构
ggplot(data_random, aes(x, y)) +
  geom_point() +
  theme_minimal() +
  labs(title = "案例3: 无结构 (随机噪音)")
```

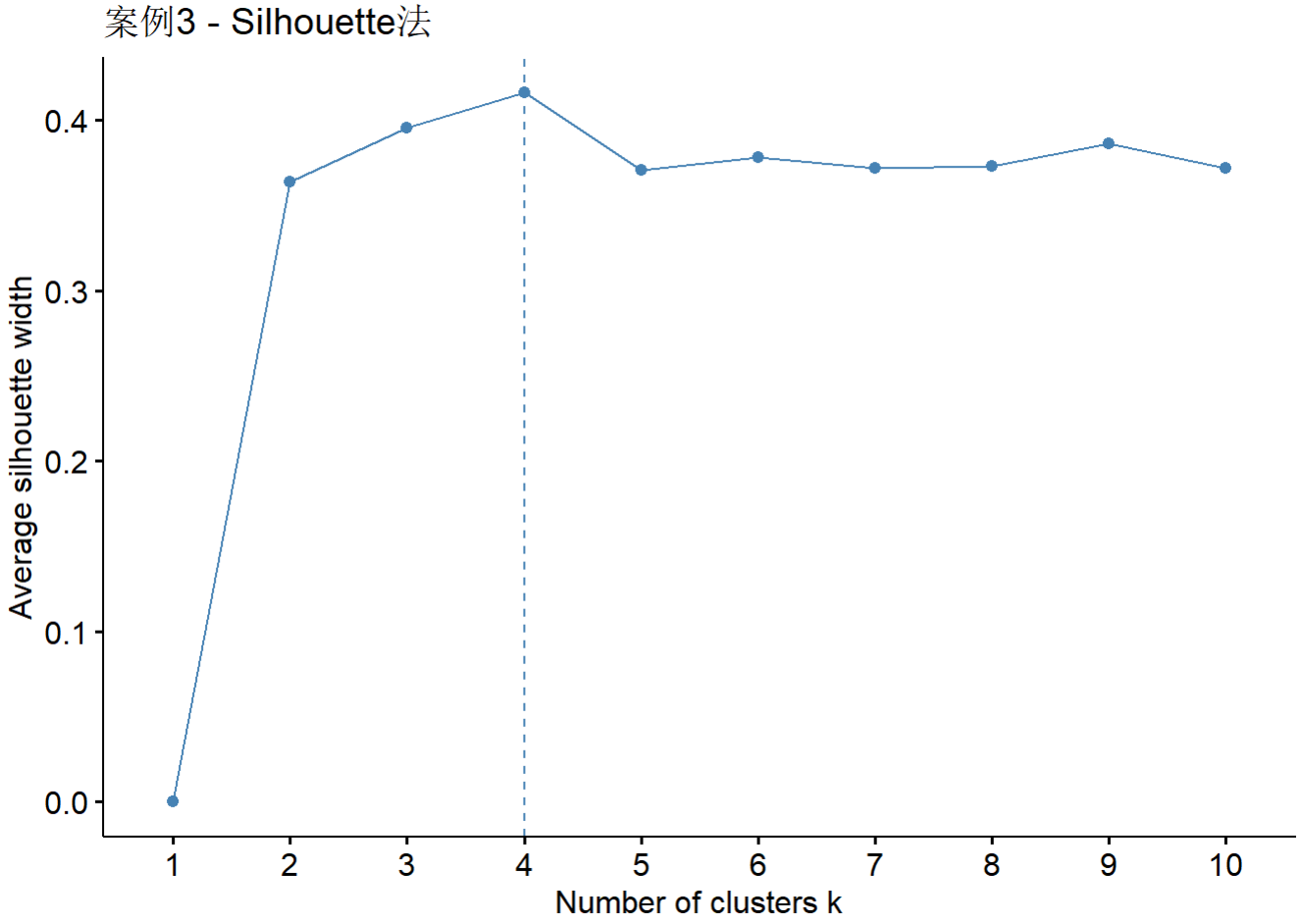
案例3: 无结构 (随机噪音)



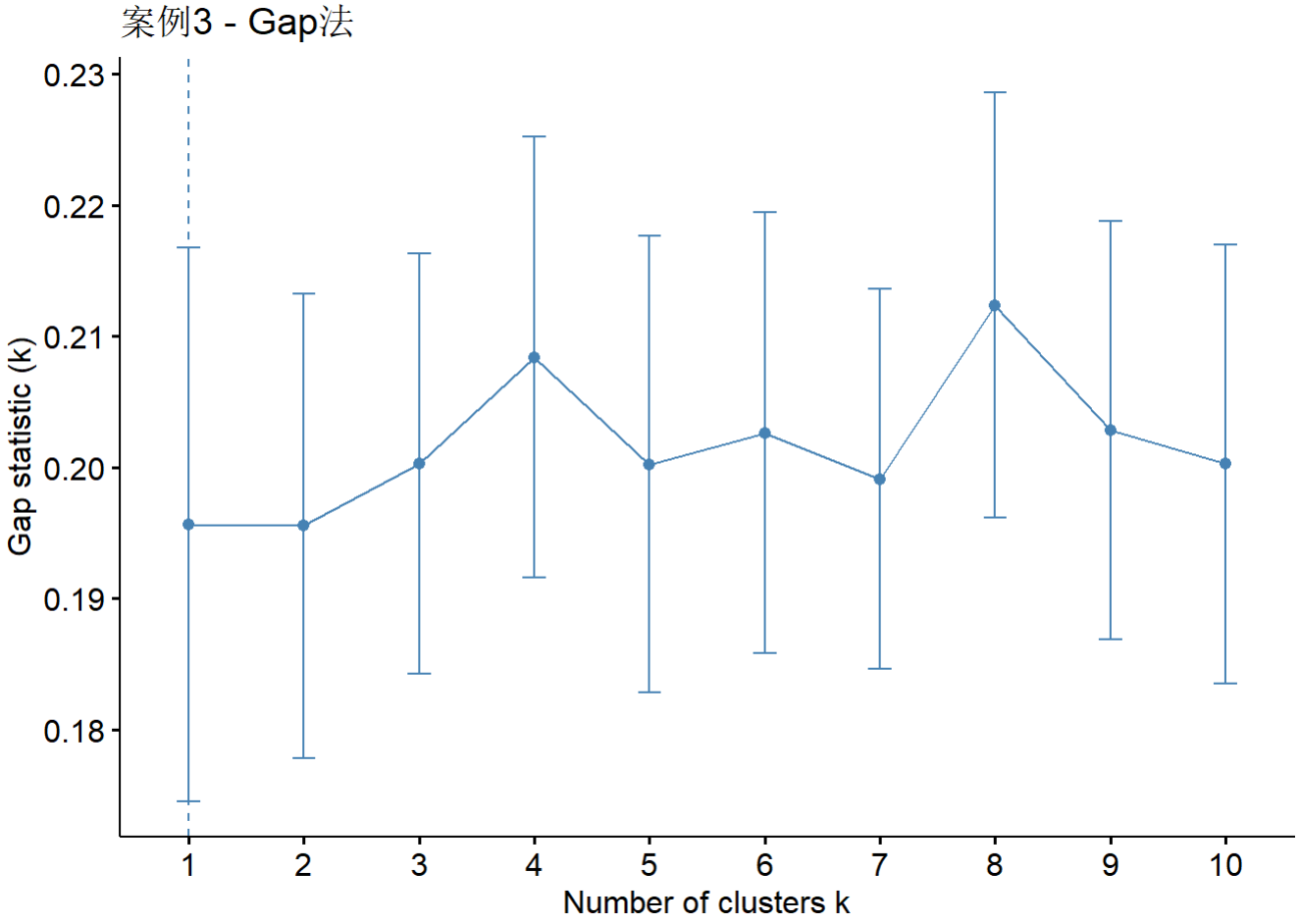

```
# 1. Elbow 法
p_elbow_3 <- fviz_nbclust(data_random, kmeans, method = "wss") +
  labs(title = "案例3 - Elbow法")
p_elbow_3
```




```
# 陷阱！新手可能还是会找一个“肘点”

# 2. Silhouette 法
p_sil_3 <- fviz_nbclust(data_random, kmeans, method = "silhouette") +
  labs(title = "案例3 - Silhouette法")
p_sil_3
```




```
# 注意看 Y 轴，所有的值都非常低！

# 3. Gap 统计量法
p_gap_3 <- fviz_nbclust(
  data_random,
  kmeans,
  method = "gap_stat",
  nboot = 50,
  nstart = 25
) +
  labs(title = "案例3 - Gap法")
p_gap_3
```




```
# 真正的答案：k=1 是最优的
```

kmeans实战

创建数据

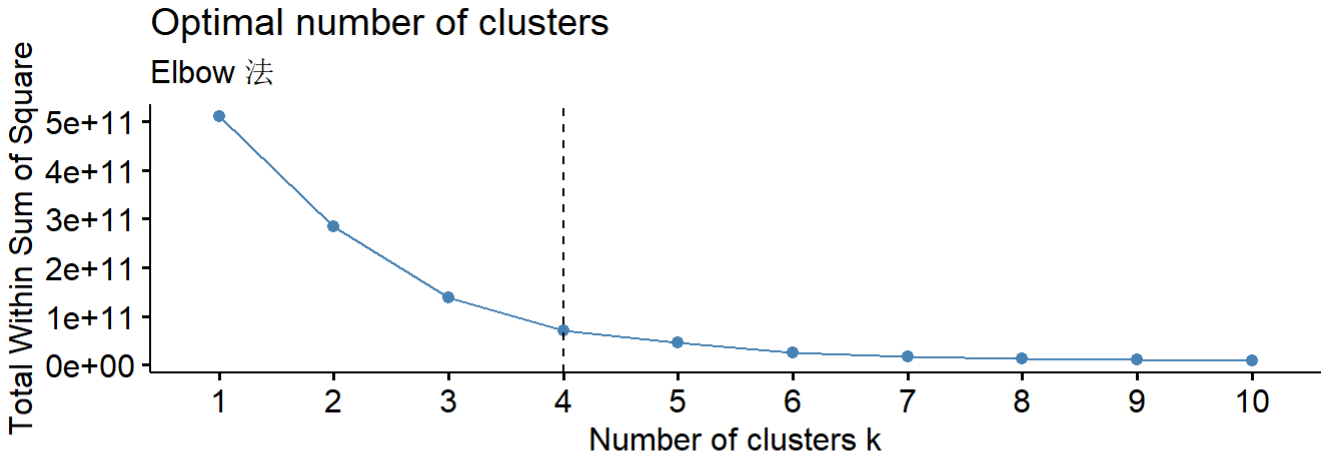
```
library(tidyverse)
library(factoextra)
# install.packages("tidyverse") # 如果你没有安装
library(tidyverse)

# 设定随机种子，确保你运行代码时能得到和我完全一样的数据
set.seed(42)

# 设定观测数
n_patients <- 940

# 创建模拟数据集
patient_data <- tibble(
  # 1. 患者ID
  Patient_ID = paste0("PID-", 1001:(1000 + n_patients)),

  # 2. 模拟 F (Frequency): 近2年总入院次数
  # 使用 "负二项分布" (rnbinom) 来模拟高度右偏的计数数据
  # 大多数患者入院1-2次，极少数患者是 "常客" (frequent flyers)
  Admission_Frequency_2Y = rnbinom(n_patients, size = 0.7, mu = 2.2) + 1,

  # 3. 模拟 R (Recency): 距离上次入院天数
  # 使用 "伽马分布" (rgamma) 来模拟右偏的时间间隔数据
  # 大多数患者是最近 (如1-100天) 出院的，少数是很久以前 (如300+天) 出院的
  Days_Since_Last_Admission = as.integer(rgamma(
    n_patients,
    shape = 1.8,
    rate = 0.012
  )),

  # 4. 模拟 M (Monetary): 近2年总医疗花费
  # 使用 "对数正态分布" (rlnorm) 并与 F (入院次数) 关联
  # 入院次数越多，花费的基础越高，同时花费本身是高度右偏的
  Total_Medical_Cost_2Y = as.numeric(
    (Admission_Frequency_2Y * rnorm(n_patients, mean = 2500, sd = 400)) + # 每次入院的基础
成本
      rlnorm(n_patients, meanlog = 8.5, sdlog = 1.2) + # 个体差异的基础花费 (偏态)
      rnorm(n_patients, mean = 1000, sd = 300) # 随机噪音
  )
)

# --- 清理数据 ---
# 确保天数和花费为正数
df <- patient_data %>%
  mutate(
    Days_Since_Last_Admission = ifelse(
      Days_Since_Last_Admission <= 0,
      1,
      Days_Since_Last_Admission
    ),
    Total_Medical_Cost_2Y = round(
```

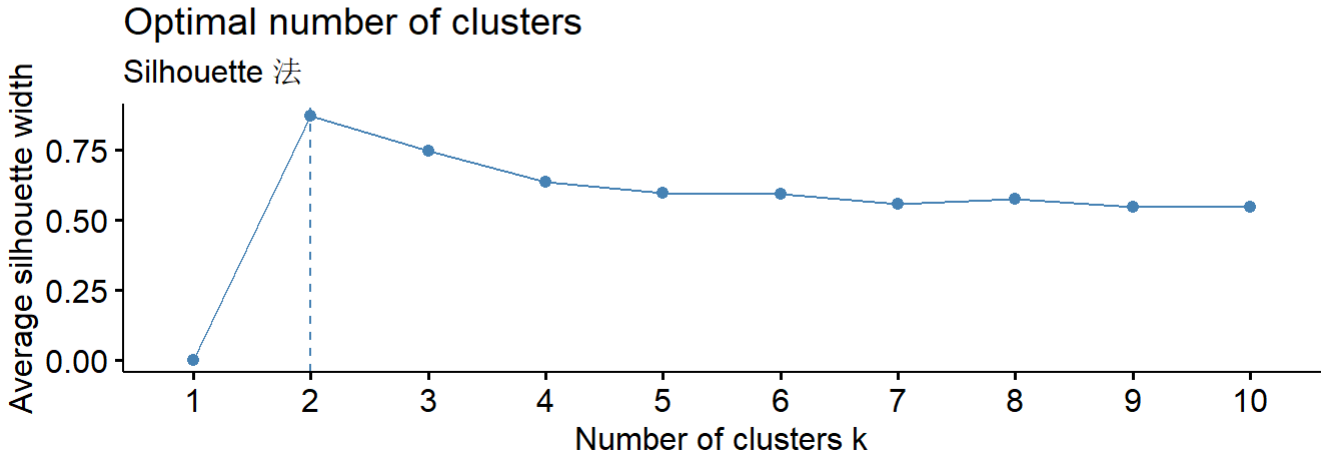
```
      ifelse(Total_Medical_Cost_2Y <= 100, 100, Total_Medical_Cost_2Y),
      2
    )
  )

head(patient_data)
```

```
# A tibble: 6 × 4
  Patient_ID Admission_Frequency_2Y Days_Since_Last_Admi…¹ Total_Medical_Cost_2Y
  <chr>                       <dbl>                 <int>                 <dbl>
1 PID-1001                        8                   347                26369.
2 PID-1002                        1                    86                16635.
3 PID-1003                        7                   242                19014.
4 PID-1004                       13                   242                42084.
5 PID-1005                        8                   127                26523.
6 PID-1006                        1                    94                29410.
# ℹ abbreviated name: ¹Days_Since_Last_Admission
```

选择k值

```
p1 <- fviz_nbclust(df[, -1], kmeans, method = "wss") +
  geom_vline(xintercept = 4, linetype = 2) +
  labs(subtitle = "Elbow 法")

p2 <- fviz_nbclust(df[, -1], kmeans, method = "silhouette") +
  labs(subtitle = "Silhouette 法")

gridExtra::grid.arrange(p1, p2, ncol = 1)
```





创建聚类任务

```
#https://mlr3book.mlr-org.com/
library(mlr3verse)
```

载入需要的程序包：mlr3

```
mlr_tasks
```

```
<DictionaryTask> with 22 stored values
Keys: ames_housing, bike_sharing, boston_housing, breast_cancer,
  california_housing, german_credit, ilpd, iris, kc_housing, moneyball,
  mtcars, optdigits, penguins, penguins_simple, pima, ruspini, sonar,
  spam, titanic, usarrests, wine, zoo
```

```
# 生成tsk
task <- as_task_clust(df, id = "patient_clust")
# 使用实际的 ID 列名（Patient_ID），将其设置为 name 角色
task$set_col_roles("Patient_ID", roles = "name")
##选择 clust.kmeans 学习器，设置聚类数为 3，训练模型：
learner = lrn("clust.kmeans", centers = 3, nstart = 25)
learner$train(task)
km = learner$model
broom::glance(km)
```

```
# A tibble: 1 × 4
          totss  tot.withinss     betweenss  iter
          <dbl>         <dbl>         <dbl> <int>
1 508923302071. 138469412283. 370453889788.     3
```

查看各个样本归入的类别

```
km_res = bind_cols(df, Class = km$cluster)
head(km_res)
```

```
# A tibble: 6 × 5
  Patient_ID Admission_Frequency_2Y Days_Since_Last_Admi…¹ Total_Medical_Cost_2Y
  <chr>                       <dbl>                  <int>                  <dbl>
1 PID-1001                        8                    347                 26369.
2 PID-1002                        1                     86                 16635.
3 PID-1003                        7                    242                 19014.
4 PID-1004                       13                    242                 42084.
5 PID-1005                        8                    127                 26523.
6 PID-1006                        1                     94                 29410.
# ℹ abbreviated name: ¹Days_Since_Last_Admission
# ℹ 1 more variable: Class <int>
```

```
# 聚类中心
km_res[, -1] %>%
  group_by(Class) %>%
  summarise(n = n(), across(.fns = mean)) %>%
  mutate(withins = km$withinss)
```

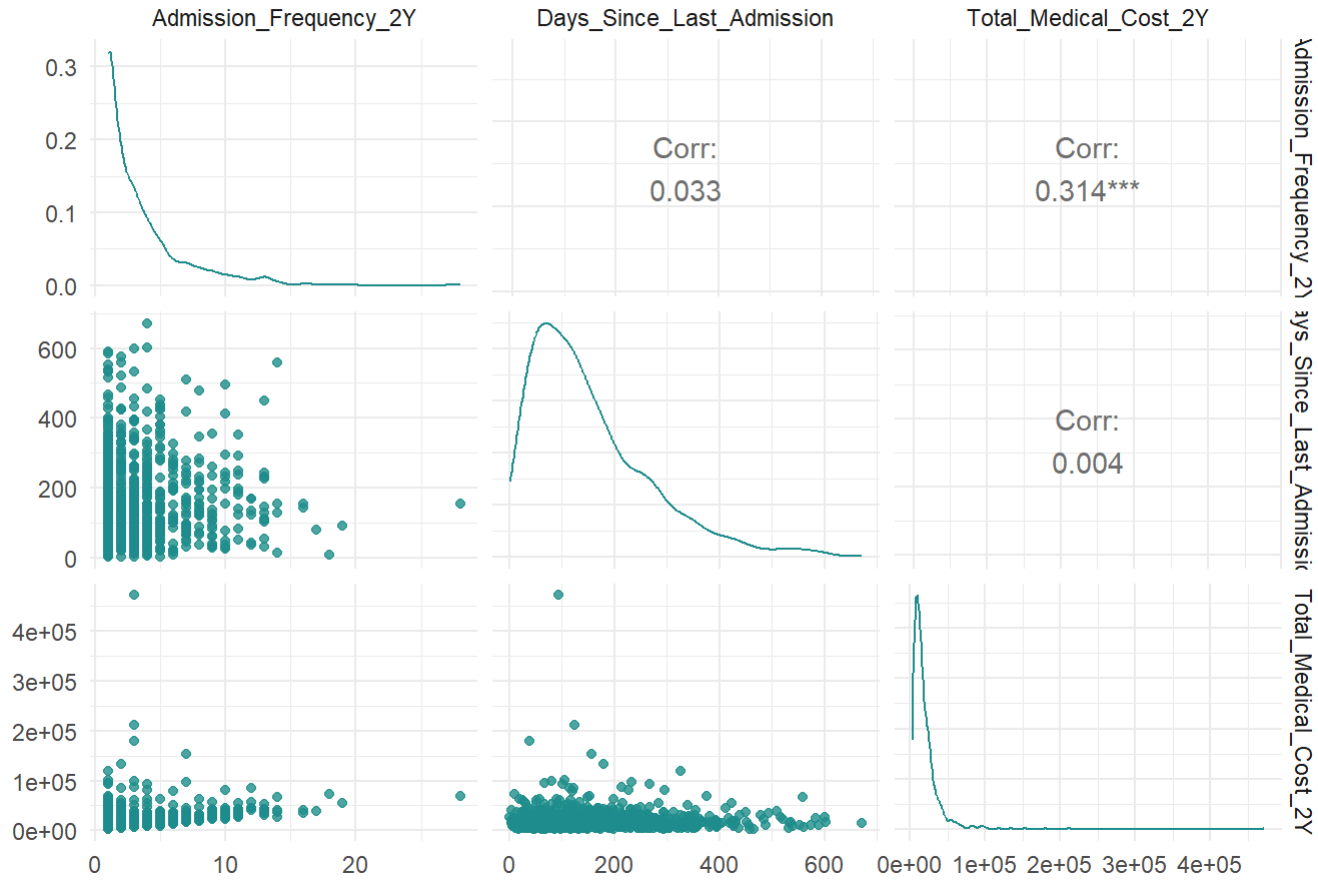
```
# A tibble: 3 × 6
  Class     n Admission_Frequency_2Y Days_Since_Last_Admission
  <int> <dbl>                  <dbl>                     <dbl>
1     1    79                   6.72                      152.
2     2   860                   2.93                      154.
3     3     1                   3                          94
# ℹ 2 more variables: Total_Medical_Cost_2Y <dbl>, withins <dbl>
```

可视化聚类结果

```
autoplot(task) + ggtitle("K-means 聚类结果可视化")
```

```
Registered S3 method overwritten by 'GGally':
  method from
  +.gg   ggplot2
```

K-means 聚类结果可视化



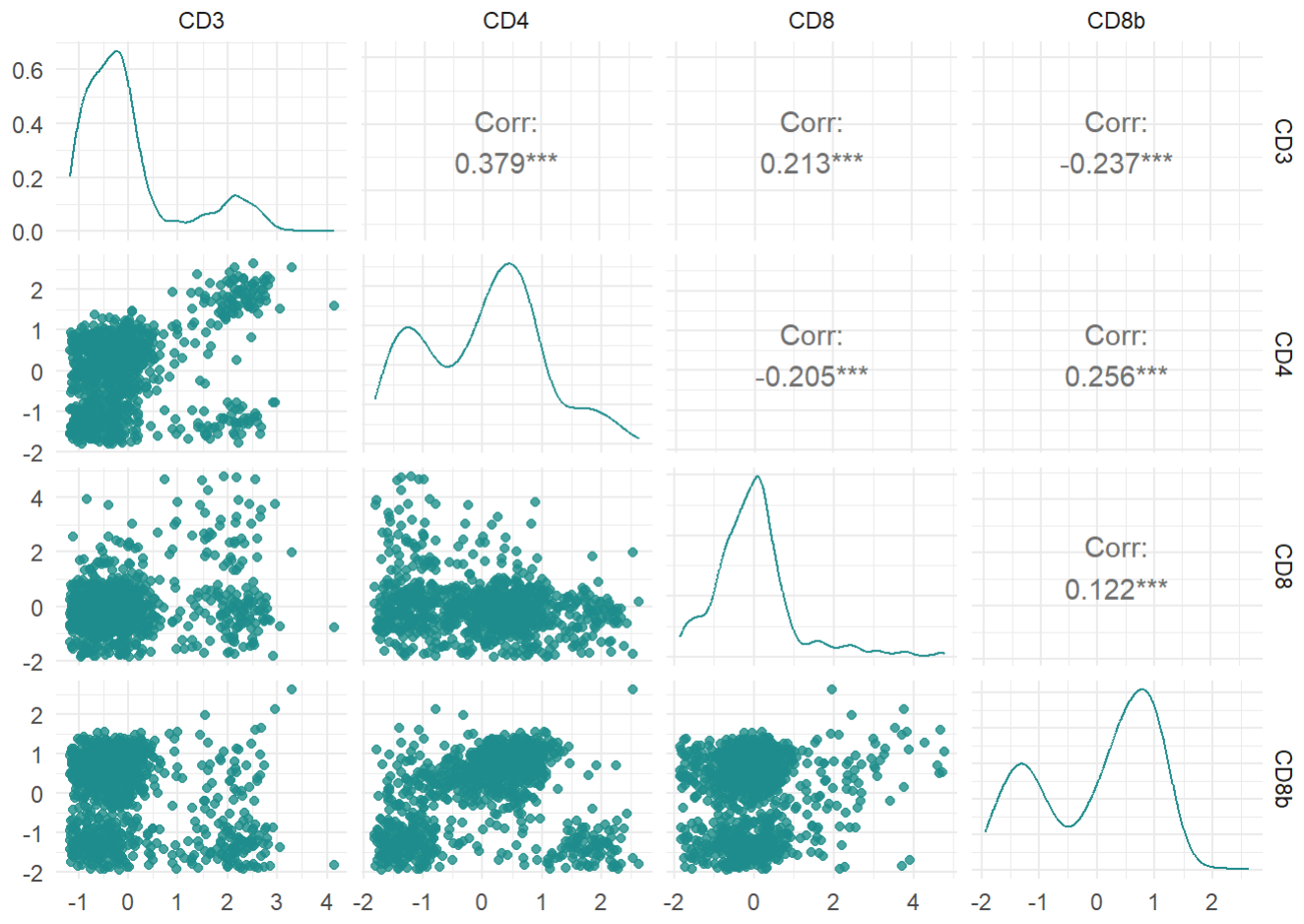
层次聚类

数据准备

```
data(GvHD, package = "mclust")
gvhdTib <- as_tibble(GvHD.control)[1:1000, ]
#对GvHD数据集进行尺度缩放
gvhdScaled <- gvhdTib %>% scale()
```

创建任务

```
task <- as_task_clust(as.data.frame(gvhdScaled))
autoplot(task)
```

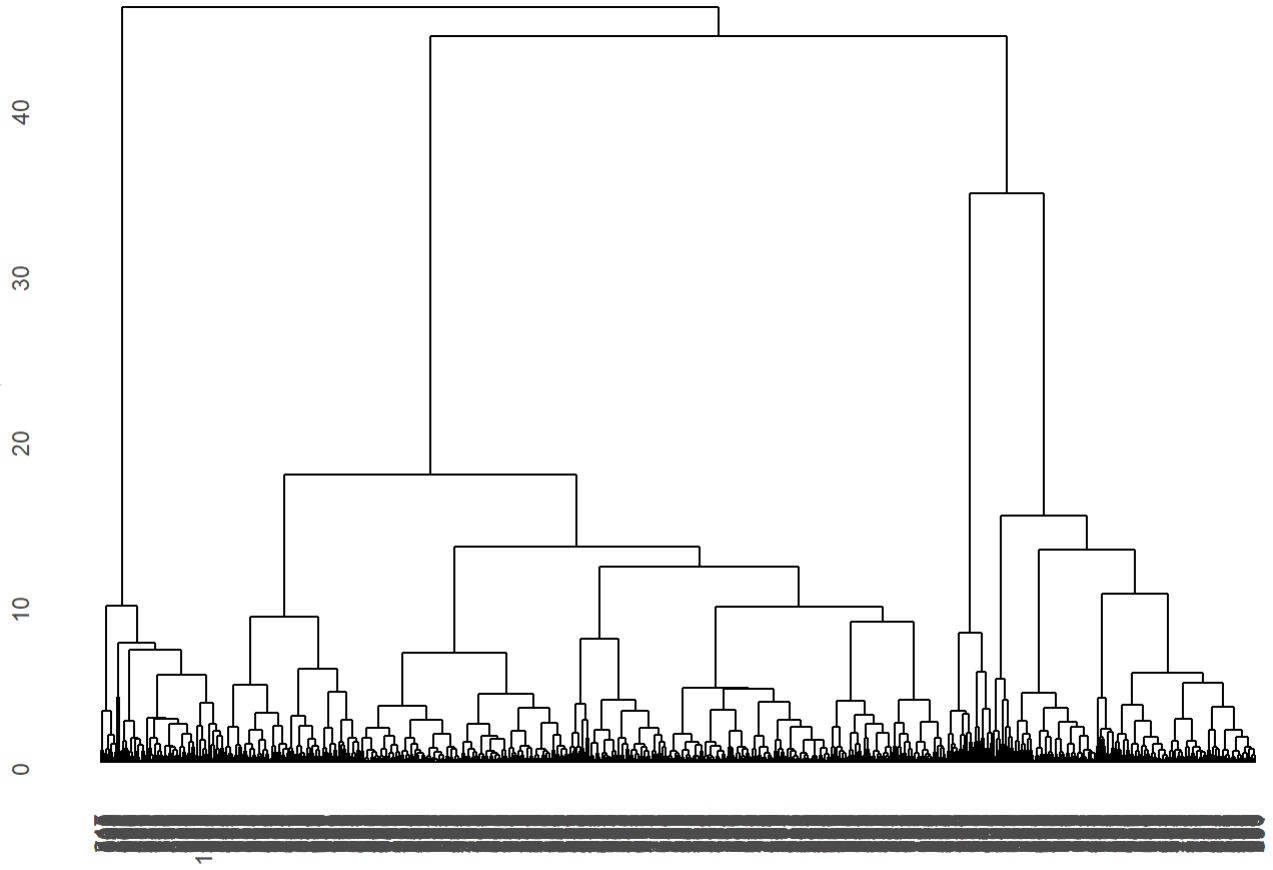


设置算法

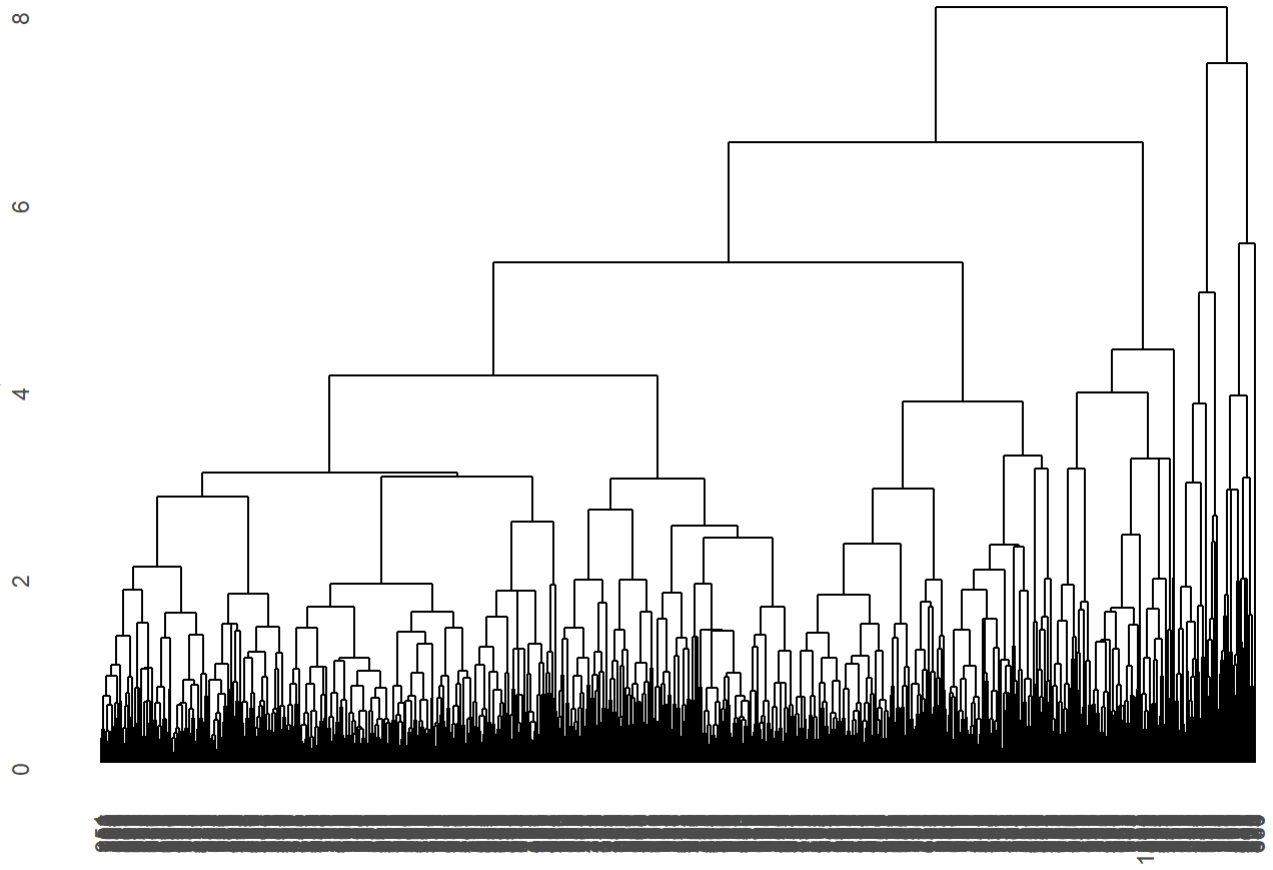
```
learner_Hclust <- lrn("clust.hclust", method = "ward.D2") #聚合聚类算法
learner_Diana <- lrn("clust.diana") #分裂聚类算法
```

训练模型

```
#聚合聚类算法
learner_Hclust$train(task)
autoplot(learner_Hclust) + theme(axis.title.y = element_text(size = 0.1))
```




```
#分裂算法
learner_Diana$train(task)
autoplot(learner_Diana) + theme(axis.title.y = element_text(size = 0.1))
```



层次聚类与K-均值算法不同，层次聚类无法预测新样本的聚类，因此无法执行类似交叉验证过程来估计不同数目聚类的性能。那我们可以使用rsmp(“insample”)，模型会在每个训练集样本上进行训练，并在相同的样本上进行测试。

评估聚类数目

```
#重新构建学习器
learner_Hclust <- lrn("clust.hclust", method = "ward.D2", k = to_tune(2, 8)) #聚合算法

future::plan("multisession")
instance_Hclust <- tune(
  tuner = tnr("grid_search"),
  task = task,
  learner = learner_Hclust,
  resampling = rsmp("insample"),
  measures = msrs(c("clust.ch", "clust.dunn", "clust.silhouette", "clust.wss")),
  terminator = trm("none")
)
```

```
Superclass MeasureClust has cloneable=FALSE, but subclass MeasureClustFPC has
cloneable=TRUE. A subclass cannot be cloneable when its superclass is not cloneable, so
cloning will be disabled for MeasureClustFPC.
Superclass MeasureClust has cloneable=FALSE, but subclass MeasureClustFPC has
cloneable=TRUE. A subclass cannot be cloneable when its superclass is not cloneable, so
cloning will be disabled for MeasureClustFPC.
```

```
Superclass MeasureClust has cloneable=FALSE, but subclass MeasureClustSil has
cloneable=TRUE. A subclass cannot be cloneable when its superclass is not cloneable, so
cloning will be disabled for MeasureClustSil.

Superclass MeasureClust has cloneable=FALSE, but subclass MeasureClustFPC has
cloneable=TRUE. A subclass cannot be cloneable when its superclass is not cloneable, so
cloning will be disabled for MeasureClustFPC.

INFO  [11:54:35.708] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:37.479] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:39.232] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:40.592] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:42.325] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:43.617] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

INFO  [11:54:45.305] [mlr3] Applying learner 'clust.hclust' on task
'as.data.frame(gvhdScaled)' (iter 1/1)

as.data.table(instance_Hclust$result)[, c(-2, -3)]

       k clust.ch clust.dunn clust.silhouette clust.wss
   <int>    <num>      <num>            <num>     <num>
1:     5 541.1113 0.02715847        0.3226502 1258.4552
2:     4 609.2865 0.05055421        0.4694021 1409.4242
3:     8 436.3213 0.02549337        0.2062768  979.6806
4:     6 493.6739 0.02943200        0.3274243 1147.1982
5:     2 350.3144 0.06756699        0.4414478 2957.7731
6:     7 457.1632 0.02549337        0.2458084 1062.1120
```

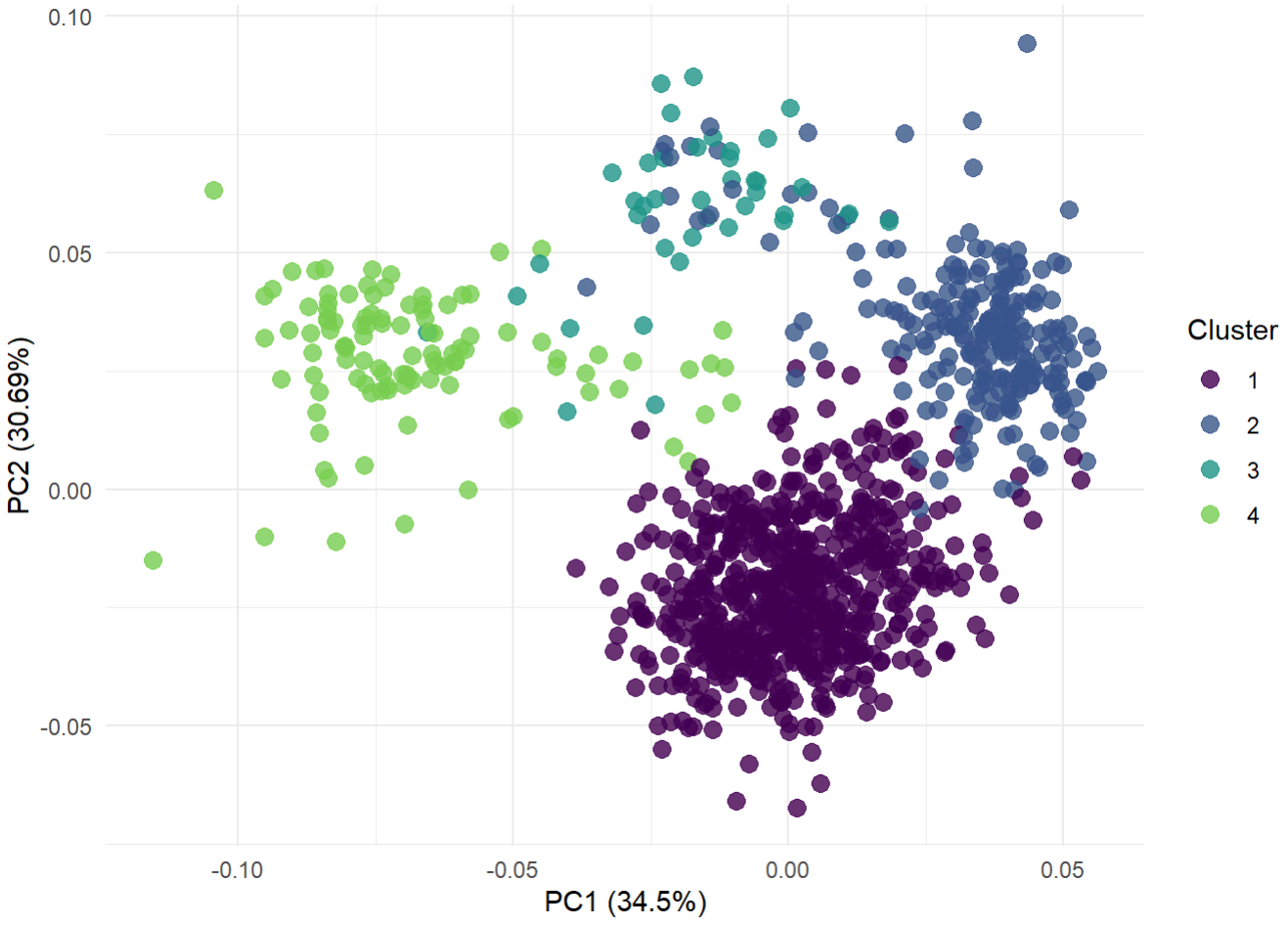
#根据性能指标，聚类数量选择4或者5比较好。通常，将无监督任务简化为定量度量可能没有用（如果没有基础事实），相反，可视化可能是评估聚类质量的更有效工具。

设定k后运行

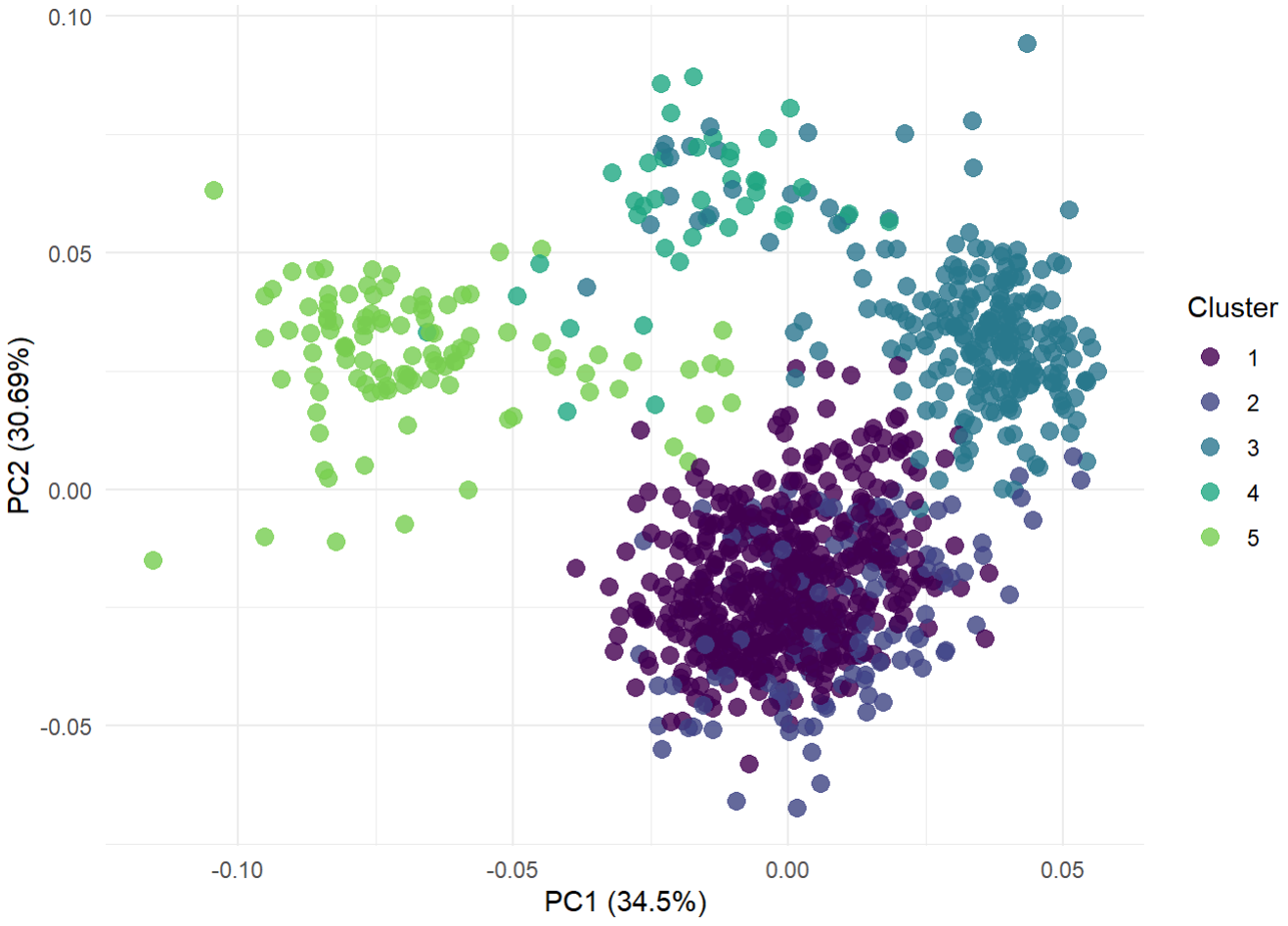
```
learner_Hclust_4 <- lrn("clust.hclust", method = "ward.D2", k = 4)
learner_Hclust_5 <- lrn("clust.hclust", method = "ward.D2", k = 5)

prediction_4 <- learner_Hclust_4$train(task)$predict(task)
prediction_5 <- learner_Hclust_5$train(task)$predict(task)
```

可视化结果

```
autoplot(prediction_4, task, type = "pca")
```




```
autoplot(prediction_5, task, type = "pca")
```

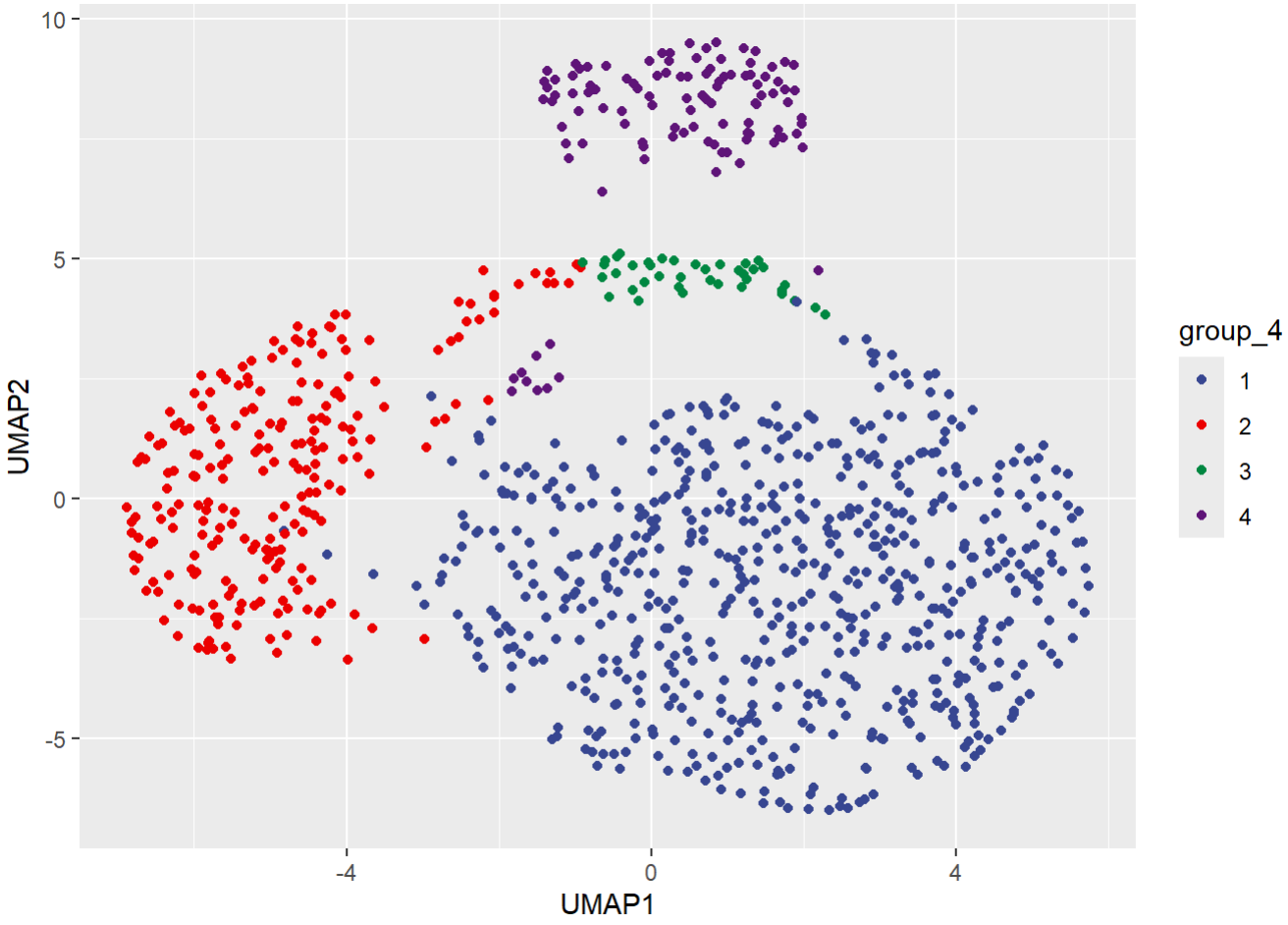


利用PCA降维后数据重叠较为严重，我们试试使用UMAP降维，再把聚类类别映射到数据集上

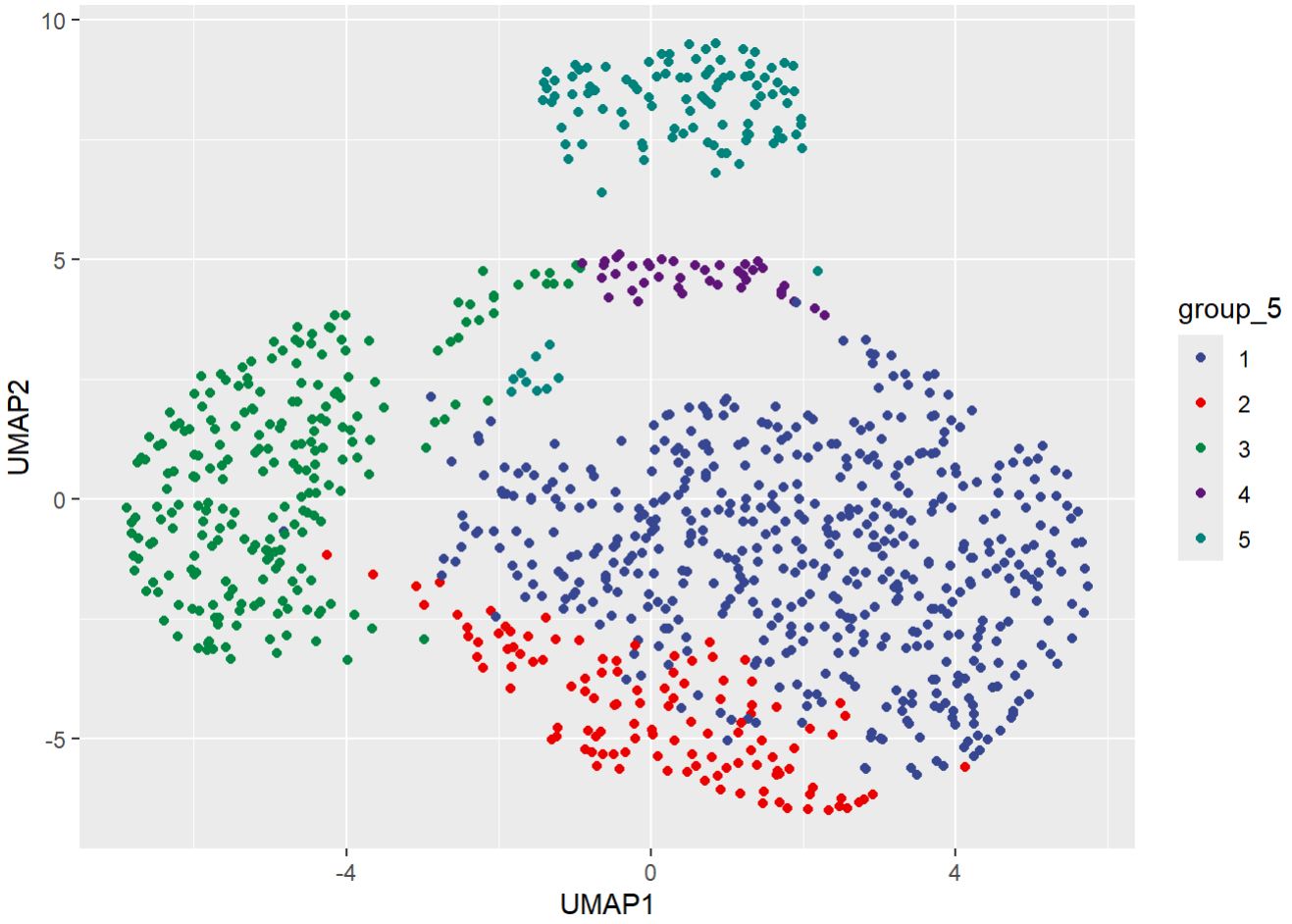
```
library(umap)
gvhd_umap <- umap(
  gvhdScaled,
  n_neighbors = 20,
  min_dist = 0.6,
  n_epochs = 500,
  verbose = TRUE
)
```

```
[2025-10-30 11:54:48.251399]  starting umap

[2025-10-30 11:54:48.406534]  creating graph of nearest neighbors

[2025-10-30 11:54:49.556659]  creating initial embedding

[2025-10-30 11:54:49.680223]  optimizing embedding

[2025-10-30 11:55:02.595285]  done
```

```
data <- data.frame(
  UMAP1 = gvhd_umap$layout[, 1],
  UMAP2 = gvhd_umap$layout[, 2],
  group_4 = factor(as.data.table(prediction_4)[[2]]),
  group_5 = factor(as.data.table(prediction_5)[[2]])
)
library(ggsci)
ggplot(data, aes(x = UMAP1, y = UMAP2, col = group_4)) +
  geom_point() +
  scale_color_aaas()
```




```
ggplot(data, aes(x = UMAP1, y = UMAP2, col = group_5)) +
  geom_point() +
  scale_color_aaas()
```

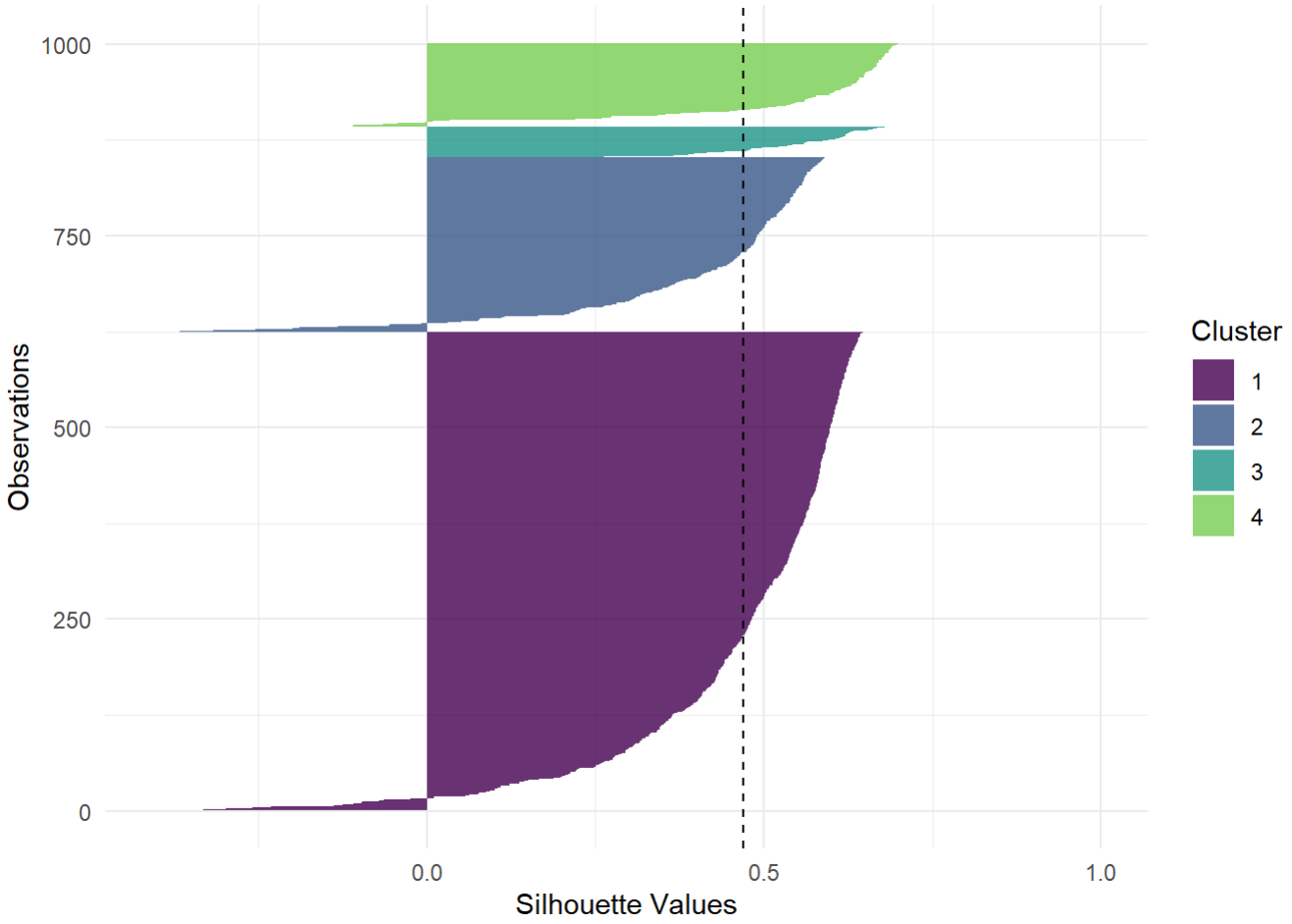


轮廓图

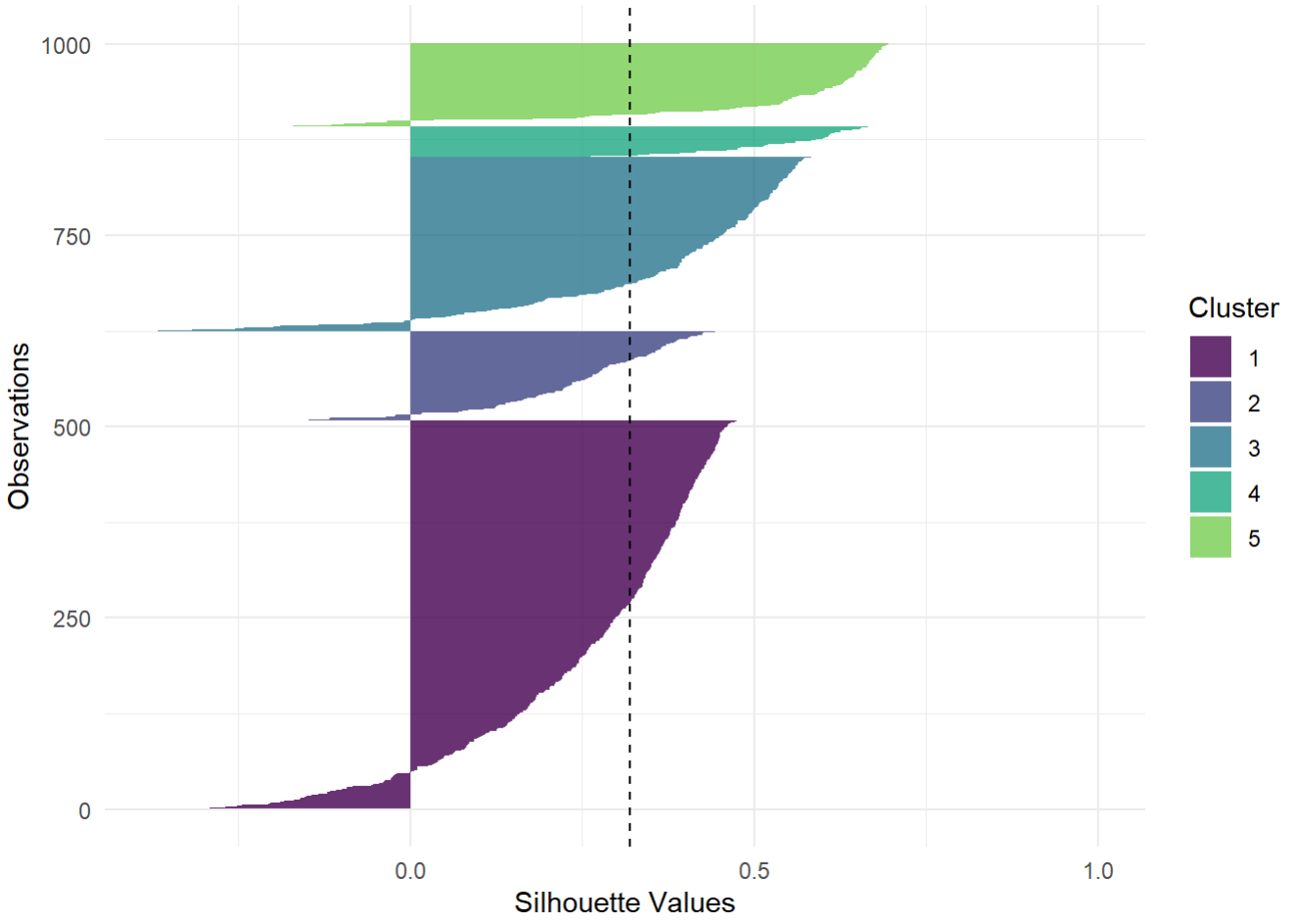
Silhouette plots 是一种用于评估聚类质量的可视化工具。在 Silhouette plots 中，每个数据点都被分配一个 Silhouette 系数，该系数表示该数据点与其所属簇内其他数据点的相似度和该数据点与最近的其他簇的数据点的不相似度之间的比率。Silhouette plots 提供了一种直观的方式来解读聚类分析的结果，帮助我们评估聚类的效果并做出相关的调整和改进。 Silhouette 系数的取值范围在 -1 到 1 之间，其中： 1 表示数据点与其所属簇内的其他数据点非常相似，并且与其他簇的数据点非常不相似； 0 表示数据点与其所属簇内的其他数据点相似度相当于与其他簇的数据点相似度； -1 表示数据点更类似于其他簇的数据点而不是其所属簇的数据点。

> 在 Silhouette plots 中，通常会显示每个数据点的 Silhouette 系数，并且会将数据点按照其所属簇进行排序。通过观察 Silhouette plots，可以帮助我们判断聚类的合理性和质量，以及确定最佳的簇数量。 从Silhouette plots图来看，在聚类类别3中有部分样本与这个类别差异较大；如果把类别3继续细分成两个类别，Silhouette会有一定的改善，但是改善不是很大。 如果给定聚类的平均轮廓值低于平均轮廓系数线（虚线），则这意味着该聚类没有被很好地定义。所以类别3没有很好的被定义。

```
autoplot(prediction_4, task, type = "sil")
```

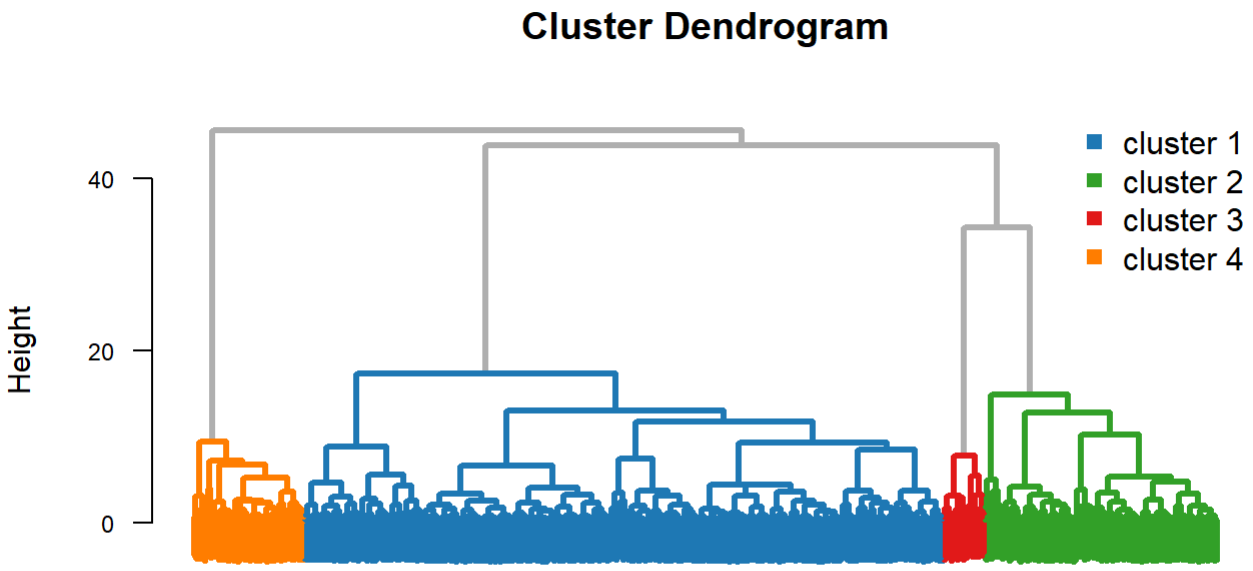



```
autoplot(prediction_5, task, type = "sil")
```



重新绘制树状图

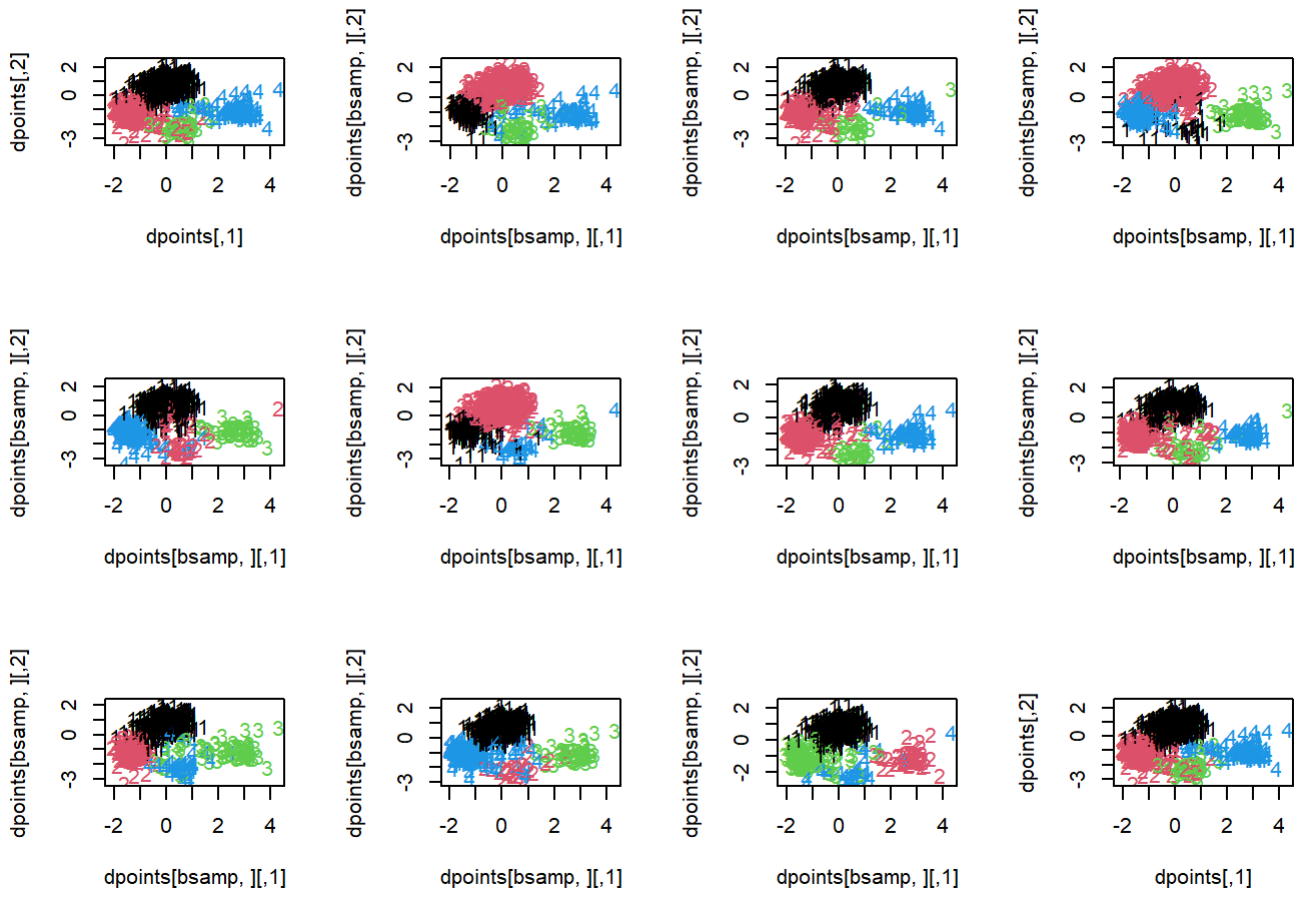
```
library(colorhcplot)
model <- learner_Hclust_4$model
gvhdcut <- cutree(model, k = 4)
gvhdTib$clusters <- gvhdcut
da <- gvhdTib %>%
  group_by(clusters) %>%
  summarize_all(mean) %>%
  round(3)

c1 <- factor(gvhdcut, levels = c(1:4), labels = paste("cluster", 1:4))
colorhcplot(model, c1)
```



Jaccard指数是一种用于衡量两个集合之间相似性的统计指标。它定义为两个集合交集的大小与它们并集大小的比值。Jaccard指数的取值范围在0到1之间，值越接近1表示两个集合的相似度越高，而值越接近0表示两个集合的相似度越低。

```
library(fpc)


#10个图合一起
par(mfrow = c(3, 4))

clustBoot <- clusterboot(
  dist(gvhdScaled, method = "euclidean"),
  B = 10,
  clustermethod = disthclustCBI,
  k = 4,
  cut = "number",
  method = "ward.D2",
  showplots = TRUE
)

boot 1

boot 2

boot 3
```

```
boot 4

boot 5

boot 6

boot 7

boot 8

boot 9

boot 10
```




```
clustBoot$bootmean

[1] 0.9700346 0.8889566 0.7255524 0.8766842
```

所有4个聚类在不同的自助采样样本中具有良好的一致性（>73%），其中三个类别一致性＞88%，这表明聚类具有很高的稳定性